

Laboratorio Tutorial

Calculadora Científica Graficadora con ANTLR

Patrón de diseño Visitor

Curso: Lenguajes de Programación y Traducción

Tema: ANTLR, árboles sintácticos y patrón Visitor

Propósito del tutorial

Construir progresivamente un pequeño lenguaje matemático utilizando ANTLR. Partiremos de una calculadora aritmética básica y la transformaremos en una calculadora científica capaz de almacenar variables, evaluar funciones matemáticas y representar gráficamente funciones.

Índice

1. Antes de comenzar	4
2. ¿Qué vamos a construir?	4
3. Recordando la calculadora del libro	5
4. Crear nuestro proyecto	6
5. Nuestra primera gramática	7
6. ¿Qué reconoce esta gramática?	8
7. Generar el Visitor	8
8. Crear nuestro Visitor	9
9. Interpretar números	10
10. Implementar suma y resta	10
11. Paréntesis	12
12. Construir el programa principal	12
13. Mostrar los resultados	13
14. Primera prueba del intérprete	14
15. Incorporar variables	14
16. Compruebe las variables	16
17. Agregar potencia	16
18. Compruebe la potencia	17
19. Funciones matemáticas	17
20. Implementar las funciones	18

21.Pruebe las funciones	19
22.Operadores unarios	19
23.Constantes matemáticas	20
24.Nuestra primera calculadora científica	21
25.Crear el comando clear	21
26.Crear el comando vars	22
27.Llegó el momento de graficar	23
28.Diseñar el comando plot	24
29.¿Qué representan las tres expresiones?	25
30.Muestrear una función	25
31.Implemente visitPlotExpr	26
32.Un problema interesante	27
33.Crear la ventana gráfica	27
34.Encontrar ymin y ymax	28
35.Transformar coordenadas	29
36.Dibujar la función	30
37.Primer gráfica	31
38.Archivo de pruebas	32
39.Explore el árbol sintáctico	32
40.Compruebe todo el lenguaje	33
41.Preguntas finales	33
42.Retos	34

43.Lista de comprobación	35
44.Reflexión final	36

1. Antes de comenzar

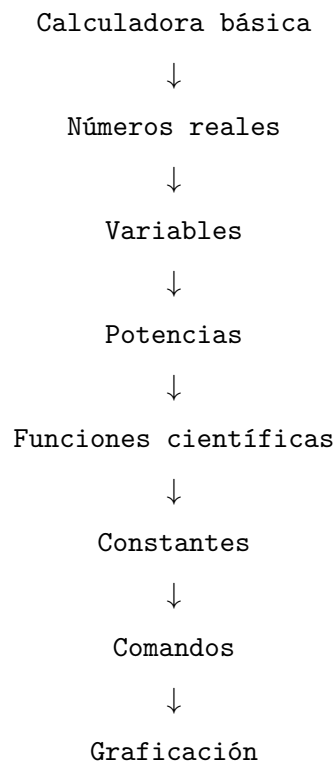
Este laboratorio está diseñado como un **tutorial**. No se espera que implemente toda la aplicación desde el principio.

La idea es avanzar en pequeños pasos.

En cada paso encontrará:

1. una explicación del concepto;
2. una modificación de la gramática o del código;
3. una prueba;
4. una pequeña actividad que deberá completar.

Al finalizar, la evolución será:



2. ¿Qué vamos a construir?

Al terminar el laboratorio queremos poder ejecutar expresiones como:

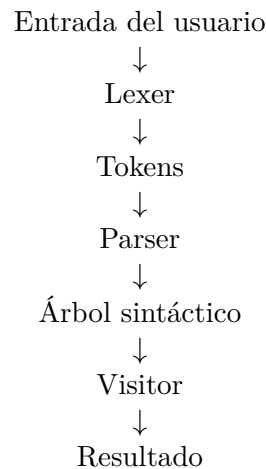
```
1 2 + 3 * 4
2
3 radio = 10
4 area = pi * radio^2
```

```
5  
6 area  
7  
8 sin(pi/2)  
9 sqrt(25)  
10 log(100)  
11  
12 plot(sin(x), -6.28, 6.28)  
13 plot(x^2, -10, 10)
```

El lenguaje tendrá, por tanto, dos características importantes:

- permitirá **evaluar expresiones matemáticas**;
- permitirá **ejecutar comandos**.

Nuestro pequeño intérprete tendrá la siguiente arquitectura:



3. Recordando la calculadora del libro

Objetivo del paso

Comprender la relación entre la gramática, el árbol sintáctico y el patrón Visitor.

La sección 4.2 del capítulo de ANTLR parte de una calculadora sencilla.

Una versión resumida de la regla para las expresiones es:

```
1 expr  
2 : expr op=('*' | '/' ) expr # MulDiv  
3 | expr op=('+' | '-' ) expr # AddSub  
4 | INT # int  
5 | ID # id  
6 | '(' expr ')' # parens  
7 ;
```

Observe elementos como:

```
1 # MulDiv
2 # AddSub
3 # int
4 # id
5 # parens
```

Estas etiquetas indican a ANTLR que genere diferentes tipos de nodos en el árbol sintáctico.

También permiten generar métodos como:

```
1 visitMulDiv(...)
2 visitAddSub(...)
3 visitInt(...)
4 visitId(...)
5 visitParens(...)
```

Esto simplifica considerablemente la implementación del intérprete.

Detente y analiza

¿Por qué cree que resulta conveniente tener un método diferente para una suma, una multiplicación y un número?
Escriba una explicación de aproximadamente tres líneas antes de continuar.

4. Crear nuestro proyecto

Cree una carpeta llamada:

```
1 ScientificCalculator
```

Inicialmente tendrá:

```
1 ScientificCalculator/
2 |
3 +-- ScientificCalc.g4
4 +-- Main.java
5 +-- ScientificEvalVisitor.java
6 +-- PlotWindow.java
7 +-- ejemplos.txt
```

Durante el laboratorio ANTLR generará otros archivos automáticamente.

Importante

Los archivos generados por ANTLR no deben modificarse manualmente.
Cada vez que se cambie la gramática se deberán volver a generar.

5. Nuestra primera gramática

Comencemos con una calculadora sencilla.

Cree:

```
1 ScientificCalc.g4
```

y escriba:

```
1 grammar ScientificCalc;
2
3 prog
4     : stat+ EOF
5     ;
6
7 stat
8     : expr NEWLINE # printExpr
9     | ID '=' expr NEWLINE # assign
10    | NEWLINE # blank
11    ;
12
13 expr
14     : expr op=('*' | '/') expr # mulDiv
15     | expr op=('+' | '-') expr # addSub
16     | NUMBER # number
17     | ID # id
18     | '(' expr ')' # parens
19     ;
20
21 MUL : '*';
22 DIV : '/';
23 ADD : '+';
24 SUB : '-';
25
26 NUMBER
27     : [0-9]+ ( '.' [0-9]+ )?
28     ;
29
30 ID
31     : [a-zA-Z_] [a-zA-Z_0-9]*
32     ;
33
34 NEWLINE
35     : '\r'? '\n'
36     ;
37
38 WS
39     : [ \t]+ -> skip
40     ;
```


6. ¿Qué reconoce esta gramática?

Antes de generar código Java debemos comprender nuestra gramática.

La regla:

```
1 NUMBER
2   : [0-9]+ ( '.' [0-9]+ )?
3   ;
```

permite reconocer:

```
1 10
2 25
3 3.14
4 100.5
5 0.25
```

La regla:

```
1 ID
2   : [a-zA-Z_] [a-zA-Z_0-9]*
3   ;
```

permite reconocer identificadores como:

```
1 a
2 radio
3 resultado
4 x1
5 variable_2
```

Ahora hazlo tú

Determine si los siguientes textos pueden ser reconocidos por la regla ID:

- variable
- x2
- 2x
- _resultado
- variable-final

Para aquellos que no puedan ser reconocidos, explique brevemente la razón.

7. Generar el Visitor

Desde la terminal, dentro de la carpeta del proyecto, ejecute:

```
1 antlr4 -no-listener -visitor ScientificCalc.g4
```

ANTLR deberá crear:

```
1 ScientificCalcLexer.java
2 ScientificCalcParser.java
3 ScientificCalcVisitor.java
4 ScientificCalcBaseVisitor.java
```

Observe especialmente:

```
1 ScientificCalcVisitor.java
```

Busque los métodos:

```
1 visitPrintExpr(...)
2 visitAssign(...)
3 visitMulDiv(...)
4 visitAddSub(...)
5 visitNumber(...)
6 visitId(...)
7 visitParens(...)
```

Detente y analiza

Compare esos métodos con las etiquetas utilizadas en la gramática.
¿Qué relación encuentra?

8. Crear nuestro Visitor

Cree:

```
1 ScientificEvalVisitor.java
```

Comience con:

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class ScientificEvalVisitor
5     extends ScientificCalcBaseVisitor<Double> {
6
7     Map<String, Double> memory = new HashMap<>();
8 }
```

Observe:

```
1 ScientificCalcBaseVisitor<Double>
```

El tipo:

```
1 Double
```

indica que los métodos del Visitor devolverán valores reales.

Esto constituye una primera diferencia frente a la calculadora del ejemplo original, que trabajaba con números enteros.

9. Interpretar números

Agregue:

```
1 @Override
2 public Double visitNumber(
3     ScientificCalcParser.NumberContext ctx) {
4
5     return Double.parseDouble(
6         ctx.NUMBER().getText()
7     );
8 }
```

¿Qué sucede aquí?

1. ANTLR ha reconocido un token `NUMBER`.
2. `getText()` obtiene el texto original.
3. `Double.parseDouble()` convierte el texto en número.

Por ejemplo:

```
1 "3.1416"
```

se convierte en:

```
1 3.1416
```

de tipo `double`.

10. Implementar suma y resta

Agregue:

```
1 @Override
2 public Double visitAddSub(
3     ScientificCalcParser.AddSubContext ctx) {
4
5     double left = visit(ctx.expr(0));
6     double right = visit(ctx.expr(1));
7 }
```

```

8   if (ctx.op.getType() == ScientificCalcParser.ADD) {
9       return left + right;
10  }
11
12  return left - right;
13 }

```

Observe cuidadosamente:

```

1 visit(ctx.expr(0))
2 visit(ctx.expr(1))

```

Estos métodos visitan los dos subárboles de la operación.

Para:

```

1 10 + 20

```

podemos imaginar:

$$\begin{array}{cc}
 & + \\
 / & \backslash \\
 10 & 20
 \end{array}$$

El Visitor primero obtiene:

$$left = 10$$

y luego:

$$right = 20$$

Finalmente ejecuta:

$$10 + 20 = 30$$

Ahora hazlo tú

Utilizando la misma estrategia, implemente:

- multiplicación;
- división.

El método generado por ANTLR se denomina:

```

1 visitMulDiv(...)

```

11. Paréntesis

Para los paréntesis podemos simplemente visitar la expresión interna:

```
1 @Override
2 public Double visitParens(
3     ScientificCalcParser.ParensContext ctx) {
4
5     return visit(ctx.expr());
6 }
```

Así:

```
1 (2 + 3)
```

devuelve el mismo resultado que:

```
1 2 + 3
```

pero los paréntesis modifican la estructura del árbol y, por tanto, la precedencia.

12. Construir el programa principal

Cree:

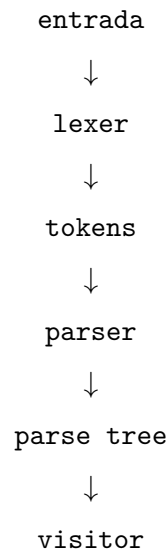
```
1 Main.java
```

con el siguiente contenido:

```
1 import org.antlr.v4.runtime.*;
2 import org.antlr.v4.runtime.tree.*;
3
4 public class Main {
5
6     public static void main(String[] args) throws Exception {
7
8         CharStream input =
9             CharStreams.fromStream(System.in);
10
11         ScientificCalcLexer lexer =
12             new ScientificCalcLexer(input);
13
14         CommonTokenStream tokens =
15             new CommonTokenStream(lexer);
16
17         ScientificCalcParser parser =
18             new ScientificCalcParser(tokens);
19
20         ParseTree tree =
21             parser.prog();
22
23         ScientificEvalVisitor visitor =
```

```
24     new ScientificEvalVisitor();
25
26     visitor.visit(tree);
27 }
28 }
```

Este programa realiza la secuencia:



13. Mostrar los resultados

Todavía debemos indicar qué hacer cuando una expresión ocupa una línea.

Agregue:

```
1 @Override
2 public Double visitPrintExpr(
3     ScientificCalcParser.PrintExprContext ctx) {
4
5     double value = visit(ctx.expr());
6
7     System.out.println(value);
8
9     return value;
10 }
```

Ahora compile su proyecto.

```
1 javac *.java
```

Ejecute:

```
1 java Main
```

Escriba:

```
1 2+2
```

La salida esperada es:

```
1 4.0
```

14. Primera prueba del intérprete

Comprueba tu implementación

Compruebe:

```
1 2+2
2 10-3
3 10*5
4 20/4
5 2+3*4
6 (2+3)*4
```

Resultados esperados:

Expresión	Resultado
2+2	4
10-3	7
10*5	50
20/4	5
2+3*4	14
(2+3)*4	20

Si:

```
1 2+3*4
```

produce 14, nuestra gramática está respetando la precedencia esperada.

15. Incorporar variables

Nuestro Visitor ya contiene:

```
1 Map<String, Double> memory = new HashMap<>();
```

Este mapa será nuestra **tabla de símbolos**.

Podemos imaginar:

Identificador	Valor
a	10.0
b	20.0
radio	5.5

Implemente:

```
1 @Override
2 public Double visitAssign(
3     ScientificCalcParser.AssignContext ctx) {
4
5     String id = ctx.ID().getText();
6
7     double value = visit(ctx.expr());
8
9     memory.put(id, value);
10
11     return value;
12 }
```

También necesitamos recuperar una variable:

```
1 @Override
2 public Double visitId(
3     ScientificCalcParser.IdContext ctx) {
4
5     String id = ctx.ID().getText();
6
7     if (memory.containsKey(id)) {
8         return memory.get(id);
9     }
10
11     System.err.println(
12         "Variable no definida: " + id
13     );
14
15     return 0.0;
16 }
```


16. Compruebe las variables

Comprueba tu implementación

Ejecute:

```
1 a = 10
2 b = 20
3 a+b
4 a*b
```

Debe obtener:

```
1 30.0
2 200.0
```

Detente y analiza

¿Qué sucedería si escribiera:

```
1 resultado + 10
```

sin haber asignado previamente un valor a **resultado**?

¿Considera adecuado devolver cero o sería mejor producir un error?

17. Agregar potencia

Ahora vamos a extender el lenguaje.

Queremos reconocer:

```
1 2^8
```

Modifique la regla:

```
1 expr
```

agregando:

```
1 <assoc=right> expr '^' expr # power
```

La regla puede quedar:

```
1 expr
2   : <assoc=right> expr '^' expr # power
3   | expr op=('*'|'/') expr # mulDiv
4   | expr op=('+'|'-') expr # addSub
5   | NUMBER # number
6   | ID # id
7   | '(' expr ')' # parens
8   ;
```

No olvide regenerar:

```
1 antlr4 -no-listener -visitor ScientificCalc.g4
```

Ahora aparecerá:

```
1 visitPower(...)
```

Implemente:

```
1 @Override
2 public Double visitPower(
3     ScientificCalcParser.PowerContext ctx) {
4
5     double base = visit(ctx.expr(0));
6     double exponent = visit(ctx.expr(1));
7
8     return Math.pow(base, exponent);
9 }
```

18. Compruebe la potencia

Comprueba tu implementación

Pruebe:

```
1 2^8
2 10^2
3 2^3+4
4 2*3^2
```

Ahora hazlo tú

Determine manualmente el resultado esperado para cada expresión antes de ejecutarla.
Compare posteriormente su respuesta con la calculadora.

19. Funciones matemáticas

Ahora queremos reconocer:

```
1 sin(x)
2 cos(x)
3 sqrt(x)
```

Agregue a la gramática:

```
1 function '(' expr ')' # functionCall
```

y cree:

```

1 function
2   : 'sin'
3   | 'cos'
4   | 'tan'
5   | 'sqrt'
6   | 'log'
7   | 'ln'
8   | 'abs'
9   | 'exp'
10  ;

```

La regla de expresiones tendrá entonces una estructura semejante a:

```

1 expr
2   : <assoc=right> expr '^' expr # power
3   | expr op=('*'|'/') expr # mulDiv
4   | expr op=('+'|'-') expr # addSub
5   | function '(' expr ')' # functionCall
6   | NUMBER # number
7   | ID # id
8   | '(' expr ')' # parens
9   ;

```

Regeneramos nuevamente:

```

1 antlr4 -no-listener -visitor ScientificCalc.g4

```

20. Implementar las funciones

Agregue:

```

1 @Override
2 public Double visitFunctionCall(
3     ScientificCalcParser.FunctionCallContext ctx) {
4
5     String function =
6         ctx.function().getText();
7
8     double value =
9         visit(ctx.expr());
10
11     switch (function) {
12
13         case "sin":
14             return Math.sin(value);
15
16         case "cos":
17             return Math.cos(value);
18
19         case "tan":
20             return Math.tan(value);

```

```
21
22     case "sqrt":
23         return Math.sqrt(value);
24
25     case "log":
26         return Math.log10(value);
27
28     case "ln":
29         return Math.log(value);
30
31     case "abs":
32         return Math.abs(value);
33
34     case "exp":
35         return Math.exp(value);
36
37     default:
38         throw new RuntimeException(
39             "Funcion desconocida: " + function
40         );
41 }
42 }
```

21. Pruebe las funciones

Comprueba tu implementación

Compruebe:

```
1 sqrt(25)
2 cos(0)
3 log(100)
4 abs(-10)
```

Algunas de estas expresiones todavía pueden generar problemas.

Por ejemplo:

```
1 abs(-10)
```

requiere reconocer números negativos.

Ese será nuestro siguiente paso.

22. Operadores unarios

Agregue:

```
1 op=('+' | '-' ) expr # unary
```

dentro de expr.

Posteriormente regenere el Visitor.

Implemente:

```
1 @Override
2 public Double visitUnary(
3     ScientificCalcParser.UnaryContext ctx) {
4
5     double value = visit(ctx.expr());
6
7     if (ctx.op.getText().equals("-")) {
8         return -value;
9     }
10
11     return value;
12 }
```

Ahora pruebe:

```
1 -10
2 abs(-10)
3 -2+5
```

23. Constantes matemáticas

Queremos escribir:

```
1 pi
2 e
```

sin definirlas como variables.

Agregue:

```
1 constant # constantExpr
```

y:

```
1 constant
2     : 'pi'
3     | 'e'
4     ;
```

Después de regenerar el Visitor, implemente:

```
1 @Override
2 public Double visitConstantExpr(
3     ScientificCalcParser.ConstantExprContext ctx) {
4
5     String constant =
6         ctx.constant().getText();
7
8     if (constant.equals("pi")) {
```

```

9      return Math.PI;
10   }
11
12   if (constant.equals("e")) {
13       return Math.E;
14   }
15
16   return 0.0;
17 }

```

24. Nuestra primera calculadora científica

Comprueba tu implementación

Pruebe:

```

1 pi
2 2*pi
3
4 sin(pi/2)
5
6 cos(0)
7
8 log(100)
9
10 ln(e)
11
12 sqrt(25)
13
14 2^8

```

Resultados aproximados:

Expresión	Resultado
sin(pi/2)	1
cos(0)	1
log(100)	2
ln(e)	1
sqrt(25)	5
2^8	256

25. Crear el comando clear

La calculadora del capítulo utiliza un mapa como memoria.

Una extensión natural consiste en permitir que el usuario borre esa memoria.

Agregue a `stat`:

```
1 | 'clear' NEWLINE # clear
```

Regeneramos ANTLR.

Luego implementamos:

```
1 @Override
2 public Double visitClear(
3     ScientificCalcParser.ClearContext ctx) {
4
5     memory.clear();
6
7     System.out.println(
8         "Memoria eliminada."
9     );
10
11     return 0.0;
12 }
```

Comprueba tu implementación

Ejecute:

```
1 a = 10
2 b = 20
3 a+b
4
5 clear
6
7 a+b
```

Observe qué ocurre después de ejecutar `clear`.

26. Crear el comando vars

Ahora crearemos nuestro propio comando:

```
1 vars
```

Agregue a la gramática:

```
1 | 'vars' NEWLINE # showVars
```

y en el Visitor:

```
1 @Override
2 public Double visitShowVars(
3     ScientificCalcParser.ShowVarsContext ctx) {
4
5     if (memory.isEmpty()) {
6         System.out.println(
7             "No hay variables definidas."
```

```
8      );
9      return 0.0;
10     }
11
12     for (Map.Entry<String, Double> entry :
13         memory.entrySet()) {
14
15         System.out.println(
16             entry.getKey()
17             + " = "
18             + entry.getValue()
19         );
20     }
21
22     return 0.0;
23 }
```

Comprueba tu implementación

Ejecute:

```
1 a = 10
2 b = 20
3 radio = 5
4
5 vars
```

27. Llegó el momento de graficar

Hasta ahora una expresión se evalúa una sola vez.

Por ejemplo:

```
1 sin(pi/2)
```

produce un único valor.

Pero para graficar:

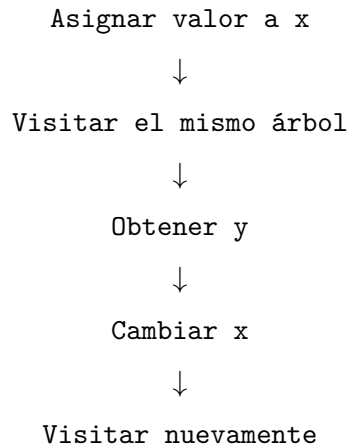
$$y = \sin(x)$$

debemos evaluar la misma expresión muchas veces.

Por ejemplo:

x	$\sin(x)$
-2	-0.909
-1	-0.841
0	0
1	0.841
2	0.909

La idea fundamental será:



28. Diseñar el comando plot

Nuestra sintaxis será:

```
1 plot(expresion, xmin, xmax)
```

Por ejemplo:

```
1 plot(sin(x), -6.28, 6.28)
```

o:

```
1 plot(x^2, -10, 10)
```

Agregue a stat:

```

1 | 'plot'
2 | '(' expr ',' expr ',' expr ')'
3 | NEWLINE
4 | # plotExpr
  
```

Regeneramos:

```
1 antlr4 -no-listener -visitor ScientificCalc.g4
```

Busque:

```
1 visitPlotExpr(...)
```

29. ¿Qué representan las tres expresiones?

Considere:

```
1 plot(sin(x), -6.28, 6.28)
```

ANTLR reconocerá tres expresiones:

```
1 ctx.expr(0)
2 ctx.expr(1)
3 ctx.expr(2)
```

que corresponden a:

Índice	Contenido
0	sin(x)
1	-6.28
2	6.28

Por tanto:

```
1 double xmin = visit(ctx.expr(1));
2 double xmax = visit(ctx.expr(2));
```

30. Muestrear una función

Supongamos que utilizamos:

```
1 int samples = 800;
```

Para cada muestra calculamos:

$$x_i = x_{\min} + i \frac{x_{\max} - x_{\min}}{N - 1}$$

En Java:

```
1 double x =
2     xmin
3     + i * (xmax - xmin)
4     / (samples - 1);
```

Luego actualizamos nuestra tabla de símbolos:

```
1 memory.put("x", x);
```

y evaluamos nuevamente el árbol:

```
1 double y = visit(ctx.expr(0));
```

Esta línea es especialmente importante.

Estamos reutilizando el árbol sintáctico.

31. Implemente visitPlotExpr

Comience con:

```
1 @Override
2 public Double visitPlotExpr(
3     ScientificCalcParser.PlotExprContext ctx) {
4
5     double xmin = visit(ctx.expr(1));
6     double xmax = visit(ctx.expr(2));
7
8     int samples = 800;
9
10    List<Double> xs =
11        new ArrayList<>();
12
13    List<Double> ys =
14        new ArrayList<>();
15
16    for (int i = 0;
17        i < samples;
18        i++) {
19
20        double x =
21            xmin
22            + i * (xmax - xmin)
23            / (samples - 1);
24
25        memory.put("x", x);
26
27        double y =
28            visit(ctx.expr(0));
29
30        xs.add(x);
31        ys.add(y);
32    }
33
34    new PlotWindow(xs, ys);
35
36    return 0.0;
37 }
```

No olvide importar:

```
1 import java.util.ArrayList;
2 import java.util.List;
```

32. Un problema interesante

¿Qué ocurre con:

```
1 plot(1/x, -5, 5)
```

cuando:

$$x = 0?$$

Java puede producir:

```
1 Infinity
2 -Infinity
3 NaN
```

Podemos comprobar un valor con:

```
1 Double.isFinite(y)
```

Ahora hazlo tú

Modifique el código de muestreo para que solamente almacene valores válidos.

Puede utilizar:

```
1 if (Double.isFinite(y)) {
2
3     xs.add(x);
4     ys.add(y);
5 }
```

¿Qué efecto tiene esta modificación sobre la gráfica de $1/x$?

33. Crear la ventana gráfica

Cree:

```
1 PlotWindow.java
```

Comience con:

```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.util.List;
4
5 public class PlotWindow extends JPanel {
6
7     private List<Double> xs;
8     private List<Double> ys;
9
10    public PlotWindow(
```

```
11         List<Double> xs,  
12         List<Double> ys) {  
13  
14         this.xs = xs;  
15         this.ys = ys;  
16  
17         JFrame frame =  
18             new JFrame(  
19                 "Scientific Calculator"  
20             );  
21  
22         frame.setDefaultCloseOperation(  
23             JFrame.DISPOSE_ON_CLOSE  
24         );  
25  
26         frame.setSize(800, 600);  
27  
28         frame.add(this);  
29  
30         frame.setVisible(true);  
31     }  
32 }
```

Todavía no dibujamos nada.

Para ello debemos sobrescribir:

```
1 paintComponent(...)
```

34. Encontrar ymin y ymax

Una gráfica necesita conocer:

$$x_{\text{mín}}, x_{\text{máx}}, y_{\text{mín}}, y_{\text{máx}}$$

Los límites horizontales vienen del comando `plot`.

Los límites verticales pueden calcularse automáticamente.

Dentro de `paintComponent`:

```
1 double xmin =  
2     xs.stream()  
3     .mapToDouble(Double::doubleValue)  
4     .min()  
5     .orElse(-1);  
6  
7 double xmax =  
8     xs.stream()  
9     .mapToDouble(Double::doubleValue)  
10    .max()  
11    .orElse(1);
```

```

12
13 double ymin =
14     ys.stream()
15     .mapToDouble(Double::doubleValue)
16     .min()
17     .orElse(-1);
18
19 double ymax =
20     ys.stream()
21     .mapToDouble(Double::doubleValue)
22     .max()
23     .orElse(1);

```

35. Transformar coordenadas

Supongamos:

$$-10 \leq x \leq 10$$

pero nuestra ventana tiene:

800 píxeles

Necesitamos una transformación.

Para el eje horizontal:

```

1 int px =
2     (int)(
3         (x - xmin)
4         / (xmax - xmin)
5         * getWidth()
6     );

```

Para el eje vertical:

```

1 int py =
2     getHeight()
3     - (int)(
4         (y - ymin)
5         / (ymax - ymin)
6         * getHeight()
7     );

```

Observe el término:

```

1 getHeight() -

```

Esto se debe a que las coordenadas gráficas de Java crecen hacia abajo.

36. Dibujar la función

Puede iniciar con:

```
1 @Override
2 protected void paintComponent(Graphics g) {
3
4     super.paintComponent(g);
5
6     Graphics2D g2 =
7         (Graphics2D) g;
8
9     if (xs.size() < 2) {
10         return;
11     }
12
13     double xmin =
14         xs.stream()
15             .mapToDouble(Double::doubleValue)
16             .min()
17             .orElse(-1);
18
19     double xmax =
20         xs.stream()
21             .mapToDouble(Double::doubleValue)
22             .max()
23             .orElse(1);
24
25     double ymin =
26         ys.stream()
27             .mapToDouble(Double::doubleValue)
28             .min()
29             .orElse(-1);
30
31     double ymax =
32         ys.stream()
33             .mapToDouble(Double::doubleValue)
34             .max()
35             .orElse(1);
36
37     for (int i = 1;
38         i < xs.size();
39         i++) {
40
41         double x1 = xs.get(i - 1);
42         double y1 = ys.get(i - 1);
43
44         double x2 = xs.get(i);
45         double y2 = ys.get(i);
46
47         int px1 =
48             (int)(
49                 (x1 - xmin)
50                 / (xmax - xmin)
```

```
51         * getWidth()
52     );
53
54     int py1 =
55         getHeight()
56         - (int)(
57             (y1 - ymin)
58             / (ymax - ymin)
59             * getHeight()
60         );
61
62     int px2 =
63         (int)(
64             (x2 - xmin)
65             / (xmax - xmin)
66             * getWidth()
67         );
68
69     int py2 =
70         getHeight()
71         - (int)(
72             (y2 - ymin)
73             / (ymax - ymin)
74             * getHeight()
75         );
76
77     g2.drawLine(
78         px1, py1,
79         px2, py2
80     );
81 }
82 }
```

37. Primera gráfica

Compile:

```
1 javac *.java
```

Ejecute:

```
1 java Main
```

Escriba:

```
1 plot(x^2,-10,10)
```

Debe aparecer una ventana con una parábola.

Posteriormente pruebe:

```
1 plot(sin(x),-6.28,6.28)
```


38. Archivo de pruebas

Cree:

```
1 ejemplos.txt
```

con:

```
1 2+2
2 2+3*4
3 (2+3)*4
4
5 a = 10
6 b = 20
7
8 a+b
9 a*b
10
11 sqrt(25)
12
13 2^8
14
15 pi
16 2*pi
17
18 sin(pi/2)
19 cos(0)
20
21 log(100)
22 ln(e)
23
24 vars
25
26 plot(x^2,-10,10)
27
28 plot(sin(x),-6.28,6.28)
```

39. Explore el árbol sintáctico

La evaluación que acabamos de implementar depende completamente del árbol sintáctico.

Analice la expresión:

```
1 sin(x) + 2*x^2
```

Identifique qué parte del árbol corresponde a:

- suma;
- función seno;
- multiplicación;

- potencia;
- identificador `x`;
- número 2.

Detente y analiza

Cuando se ejecuta el Visitor:

```
1 visit(ctx.expr())
```

¿se evalúa una cadena de texto o se visita una estructura de árbol?
Explique su respuesta.

40. Compruebe todo el lenguaje

La calculadora debe poder procesar:

```
1 radio = 10
2
3 area = pi * radio^2
4
5 area
6
7 angulo = pi/4
8
9 sin(angulo)
10 cos(angulo)
11
12 vars
13
14 plot(sin(x), -6.28, 6.28)
15
16 plot(x^2, -10, 10)
```

Si su programa puede interpretar este conjunto de instrucciones, ya dispone de un pequeño lenguaje matemático.

41. Preguntas finales

Responda las siguientes preguntas.

1. ¿Cuál es la responsabilidad del Lexer?
2. ¿Cuál es la responsabilidad del Parser?
3. ¿Qué función cumplen las etiquetas como `#addSub` o `#functionCall`?
4. ¿Qué ventaja ofrece el patrón Visitor?

5. ¿Qué representa la tabla de símbolos?
6. ¿Por qué la variable `x` cambia continuamente durante una gráfica?
7. ¿Por qué podemos evaluar el mismo árbol sintáctico varias veces?
8. ¿Qué sucede cuando se intenta graficar una función con una discontinuidad?
9. ¿Qué modificaciones serían necesarias para implementar funciones con dos argumentos?
10. ¿Por qué la calculadora desarrollada puede considerarse un lenguaje de dominio específico?

42. Retos

Reto

Reto 1 – Nuevas funciones

Agregue:

```
1 asin(x)
2 acos(x)
3 atan(x)
4 floor(x)
5 ceil(x)
```

Reto

Reto 2 – Funciones con dos argumentos

Diseñe una extensión para:

```
1 pow(2,8)
2 max(10,25)
3 min(10,25)
```

Antes de programar, escriba primero la regla de la gramática que necesitaría.

Reto

Reto 3 – Rango vertical

Amplíe:

```
1 plot(expr,xmin,xmax)
```

a:

```
1 plot(expr,xmin,xmax,ymin,ymax)
```

Reto**Reto 4 – Varias funciones**

Diseñe una sintaxis para poder escribir:

```
1 plot(sin(x),cos(x),-6.28,6.28)
```

No modifique primero el código Java.

Comience diseñando la gramática.

Reto**Reto 5 – Definir funciones**

¿Cómo podría evolucionar nuestro lenguaje para permitir?

```
1 f(x) = x^2 + 2*x + 1
2
3 f(5)
4
5 plot(f(x),-10,10)
```

Realice inicialmente solamente el diseño sintáctico.

43. Lista de comprobación

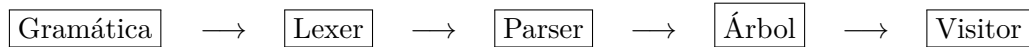
Antes de finalizar revise que su proyecto soporte:

- ☐ números reales;
- ☐ suma;
- ☐ resta;
- ☐ multiplicación;
- ☐ división;
- ☐ paréntesis;
- ☐ variables;
- ☐ potencia;
- ☐ operadores unarios;
- ☐ constantes π y e ;
- ☐ funciones científicas;
- ☐ comando `clear`;
- ☐ comando `vars`;
- ☐ comando `plot`;
- ☐ visualización gráfica.

44. Reflexión final

Durante este laboratorio comenzamos con una gramática muy pequeña para expresiones aritméticas y progresivamente construimos un lenguaje con más capacidades.

La arquitectura final puede resumirse como:



La idea central del ejercicio es comprender que:

La gramática define la sintaxis.
El Visitor implementa la semántica.

Gracias a esta separación podemos extender el lenguaje sin introducir toda la lógica de la aplicación dentro de la gramática.

Esta misma estrategia constituye la base para construir sistemas más complejos como:

- intérpretes;
- compiladores;
- traductores;
- analizadores de código;
- lenguajes de consulta;
- lenguajes de dominio específico.

La calculadora científica desarrollada en este laboratorio es, por tanto, un primer ejemplo de construcción de un pequeño **DSL matemático**.